

# Code review — mostenire si polimorfism

Runde de review pe proiectul tau, cea mai noua sus.

## Runda 1 — 2026-09-16 — sha 6444afb

Ai livrat `Admin`, lista polimorfica, `tip()` / `eticheta()`, supraincercarea `raport(...)` si provocarea 7 cu campul `rol`. Partea de mostenire e facuta corect si se vede ca ai inteles la ce foloseste `super(...)`.

Proiectul **nu compileaza** pe commit-ul pushat, iar `UserRepository` are doua cai prin care iti sterge date din `users.txt`. Le-am reprodus pe amandoua, nu le-am dedus.

## Critice

### B1 Proiectul nu compileaza

`src/app/App.java:135`, `:136`, `:137`

**CUM E ACUM:** `javac` pe commit-ul 6444afb:

```
src/app/App.java:135: error: rol has protected access in Student
    System.out.println(sTest.rol);
                        ^
src/app/App.java:136: error: rol has protected access in User
src/app/App.java:137: error: rol has protected access in Student
3 errors
```

**DE CE:** `protected` in Java deschide doua porti, nu una: campul e vizibil (1) din orice clasa aflata in **acelasi pachet** si (2) din **subclasele** lui, oriunde ar fi ele. `App` e in pachetul `app`, iar `User` si `Student` sunt in `app.users` — pachete diferite, deci prima poarta e inchisa. Si `App` nu extinde

nimic din `app.users`, deci e inchisa si a doua. Nu conteaza ca `App` e „in acelasi proiect” sau „in acelasi folder `src`” — Java se uita strict la `package`.

Mai e un amanunt care te-ar fi lovit chiar daca mutai clasa: accesul prin mostenire iti da voie sa citesti `rol` doar printr-o referinta de **tipul tau** ( `this.rol` ), nu prin `u.rol` unde `u` e declarat `User`.

**FIX:** muta cele trei linii intr-o clasa din `app.users`, ca sa fii in acelasi pachet. Am rulat asa si da exact ce cerea exercitiul 7:

```
STUDENT
USER
STUDENT
```

## B2 Un Admin salvat dispare la urmatoarea pornire

`src/app/users/UserRepository.java:48` (switch-ul din `loadData`) · `src/app/App.java:129`

**CUM E ACUM:** ruleaza de doua ori la rand — run 1 `addUser(admin)`, run 2 pornire normala:

```
--- run 1: adaug un Admin ---    -> in users.txt apare ADMIN,010,...
--- run 2: pornesc din nou ---
type nout found
STUDENT,001,...
STUDENT,002,...
TEACHER,007,...
TEACHER,008,...
--- run 2 face save() (App.java:129) ---
-> linia ADMIN nu mai exista in fisier
```

**DE CE:** la **scriere** exista polimorfism — chemi `u.descriere()` pe o referinta `User`, iar obiectul isi alege singur varianta si pune „ADMIN” in fata. La **citire** nu exista polimorfism, pentru ca nu exista inca niciun obiect: tu esti cel care decide ce clasa construiesti, si o decizi cu un `switch` scris de mana. `switch`-ul ala e harta inversa a lui `descriere()` si trebuie tinut sincronizat manual — asta e pretul serializarii pe text. Cand ai adaugat `Admin`, ai completat jumatatea de scriere ( `Admin.descriere()` ) si ai uitat jumatatea de citire.

Si ramura `default` doar printeaza — nu opreste nimic si nu pastreaza nimic. Deci linia e citita, aruncata, iar la primul `save()` lista din memorie (fara admin) se scrie **peste** fisier. Mesajul `type nout found` e singurul semn ca tocmai ai pierdut un rand.

**FIX:** `case "ADMIN": this.users.add(new Admin(line)); break;`

Cauta singur regula, nu linia: orice subclasa noua pe care o adaugi de acum incolo are nevoie de doua modificari, nu de una — `descriere()` in clasa si `case` in `loadData` .

### B3 🚫 O singura linie stricata in users.txt sterge restul fisierului

`src/app/users/UserRepository.java:42 - :60` (unde e pus `try` ) · `:85` ( `PrintWriter` )

**CUM E ACUM:** am stricat a doua linie din `users.txt` (7 campuri in loc de 8) si am pornit programul. Fisierul avea 4 linii inainte:

```
java.lang.ArrayIndexOutOfBoundsException: Index 7 out of bounds for length 7
    at app.users.Student.<init>(Student.java:28)
    at app.users.UserRepository.loadData(UserRepository.java:49)
--- ce s-a incarcat in memorie ---
STUDENT,001,ana@ex.com,p1,2026-09-01,2024,Ionescu,20
--- save() ---
-> users.txt are acum 1 linie
```

Trei linii bune, dintre care doua perfect valide, s-au dus definitiv.

**DE CE:** `try` e pus in jurul **intregii bucle** `while` , nu in jurul unei iteratii. Cand linia 2 arunca exceptia, executia sare afara din `while` — nu la urmatoarea linie, ci in afara buclei cu totul. Liniile 3 si 4 nu sunt nici macar citite. Apoi `catch` doar prindeaza si programul isi vede de treaba ca si cum totul ar fi bine: constructorul se termina, repo-ul porneste cu lista trunchiata, si nimic nu spune ca lipsesc date.

Partea care face paguba permanenta e `new PrintWriter(FILE_PATH)` din `save()` : constructorul asta **trunchiaza fisierul la zero** inainte sa scrii ceva in el. Deci fisierul bun de pe disc e sters si inlocuit cu ce ai in memorie — o lista incompleta. Cele doua bug-uri se compun: unul pierde date in RAM, celalalt le sterge de pe disc.

**FIX:** muta `try` -ul **inauntrul** buclei, pe o singura linie, ca o linie stricata sa fie sarita, nu sa opreasca citirea:

```

while (scanner.hasNextLine()) {
    String line = scanner.nextLine().trim();
    try {
        ...
    } catch (Exception e) {
        System.out.println("Skipping invalid line: " + line);
    }
}

```

#### B4 🚫 Doua cerinte din fisa lipsesc

src/app/users/UserRepository.java · src/app/App.java:131

#### CUM E ACUM:

- exercitiul 6 — `numaraStudenti()` nu exista nicaieri ( `grep` = 0 rezultate). Si in `App` cauti doar `"11"`, un id inexistent; cazul „id care exista si ii afisezi creditele” nu e apelat, desi l-ai scris in `cautaUserDupaId`. Ramura aia de cod n-a rulat niciodata.
- exercitiul 8 — `ClassCastException` nu apare deloc in proiect.

**DE CE CONTEAZA:** exercitiul 8 e singurul din fisa in care vezi cu ochii tai diferenta dintre ce stie **compilatorul** si ce stie **masina virtuala**. Compilatorul accepta `(Student) u` fiindca se uita doar la arborele de mostenire: `Student` e sub `User`, deci conversia e *posibila*. Daca e si *adevarata* se afla abia la rulare, cand JVM-ul se uita la obiectul real si gaseste un `Teacher`. De asta `String x = (String) u;` nici nu compileaza — acolo compilatorul are destula informatie ca sa stie ca e imposibil.

Fara exercitiul asta, `instanceof` ramane pentru tine o formalitate pe care o scrii din obisnuinta, nu o plasa de care chiar ai nevoie.

**FIX:** scrie ambele bucati si ruleaza-le; noteaza mesajul exact al exceptiei.

## Importante

#### M1 🟡 Exerciitiile 2 si 3 sunt amestecate in aceeași buclă

src/app/App.java:110 – :124

Exercitiul 2 avea o regula explicita: in bucla, **niciun if si niciun cast**. Tu ai pus si cele trei `if`-uri de la exercitiul 3 in aceeași bucla, deci bucla „curata” — cea care demonstreaza late binding singura, fara

ajutor — nu exista in cod. Doua bucle separate, una dupa alta.

## M2 🟡 `nume` e scris de trei ori

`src/app/users/Student.java:9` · `Teacher.java:6` · `Admin.java:6`

Toate trei subclasele au acelasi camp `nume`, cu acelasi getter. Consecinta practica: pe o referinta `User` nu poti scrie `u.getNum()` — de-aia ai fost nevoit sa faci cast si la exercitiul 3, chiar si pentru nume. Ce e comun urca in superclasa; ce diferentiaza (credite / salariu / conturi) ramane jos.

## M3 🟡 `cautaUserDupaId` face doua treburi, si rezultatul e aruncat

`src/app/users/UserRepository.java:64` — `:76` · `src/app/App.java:131`

Metoda si cauta, si **afiseaza**. Iar in `App` chemi `userRepo.cautaUserDupaId("11")` fara sa pui rezultatul nicaieri — `Optional`-ul pe care l-ai construit se pierde. Cine cauta, intoarce; cine afiseaza, afiseaza. Altfel nu poti refolosi metoda nicaieri unde nu vrei si print.

## M4 🟡 `Scanner` nu e inchis

`src/app/users/UserRepository.java:43`

`Scanner scanner = new Scanner(file);` fara `try (...)`. Fisierul ramane deschis pana cand trece garbage collectorul. Pe Windows — unde lucrezi tu — asta chiar poate face ca `save()` de mai tarziu sa nu poata rescrie fisierul.

---

## Cleanups

- **C1** 🟢 `UserRepository.java:3` — `import javax.swing.text.html.Option;` e un import fantoma pus de autocomplete; nu e folosit si nici nu e `Optional`.
- **C2** 🟢 `User.java:71` — lipseste spatiul dupa `]`: iese `[STUDENT]ana@ex.com`, fisa cerea `[STUDENT] ana@ex.com`.
- **C3** 🟢 `UserRepository.java:15` — `FILE_PATH` e scris cu MAJUSCULE, deci il gandesti ca pe o constanta; fa-l `private static final String`.
- **C4** 🟢 `UserRepository.java:54` — typo: `"type nout found"`.
- **C5** 🟢 `App.java:36` — `:93` si `UserRepository.java:96` — `:102` — ~60 de linii de cod comentat. Istoricul il tine git, nu fisierul.
- **C6** 🟢 `App.java:4` — `:9` — importurile `Vehicul`, `Masina`, `Barca`, `Avion` si `LocalDate` nu mai sunt folosite.

- **C7** 🟢 `App.java:115` , `:118` , `:121` — același cast scris de două ori pe linie; o variabilă locală în `if` îl face o singură dată.
- **C8** 🟢 `App.java:105` – `:107` — `List.of(...)` urmat de `addAll` e un ocol; `new ArrayList<>(List.of(s1, s2, t1, t2, a1))` .
- **C9** 🟢 `.idea/` și `Mostenire-Polimorfism.iml` sunt urmărite în git (4 fișiere), deși `.gitignore` e scris pentru IntelliJ. Sunt setări de pe calculatorul tau.
- **C10** 🟢 `User.java:15` , `Student.java:25` , `Teacher.java:12` — `String arr[]` e sintaxa de C; în Java se scrie `String[] arr` . Tipul aparține tipului, nu variabilei.

## Before / after (critice)

| #  | Cum e acum                                                                                                             | Cum ar trebui                                                                                                              |
|----|------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------------|
| B1 | <code>System.out.println(u.rol);</code><br>în <code>App</code> , pachetul <code>app</code>                             | aceleași linii într-o clasă din pachetul <code>app.users</code>                                                            |
| B2 | <code>switch</code> doar cu <code>case</code><br><code>"STUDENT"</code> și <code>case</code><br><code>"TEACHER"</code> | <code>case "ADMIN":</code><br><code>this.users.add(new</code><br><code>Admin(line)); break;</code>                         |
| B3 | <code>try { while (...) { ... } }</code><br><code>} catch (Exception e)</code>                                         | <code>while (...) { try { ... } }</code><br><code>catch (Exception e) {</code><br><code>continue; } }</code>               |
| B3 | <code>catch (Exception e) {</code><br><code>e.printStackTrace(); }</code>                                              | <code>catch (Exception e) {</code><br><code>System.out.println("Skipping</code><br><code>invalid line: " + line); }</code> |
| B4 | <code>cautaUserDupaId("11")</code> ,<br>rezultat aruncat                                                               | caută și un id care există,<br>folosește ce întoarce                                                                       |
| B4 | <code>numaraStudenti()</code> nu există                                                                                | <code>public int numaraStudenti()</code><br>peste <code>users</code>                                                       |

## Q&A — raspunde in scris, doua-trei randuri fiecare

1. La `save()` , `u.descriere()` alege singur varianta corecta pentru fiecare obiect. La `loadData()` esti nevoit sa scrii un `switch` de mana. De ce nu poate polimorfismul sa te ajute si la citire?
2. In B3, `catch` a prins exceptia si programul a mers mai departe „fara eroare”, dar au disparut trei linii din fisier. Unde anume, in codul tau, s-a transformat pierderea din memorie in pierdere de pe disc?
3. `App` e in pachetul `app` , `Student` in `app.users` . Daca `App extends Student` , ar compila `System.out.println(u.rol);` unde `User u = sTest;` ? Argumenteaza.



MyCodeSchool

Material didactic MyCodeSchool

[mycodeschool.ro](https://mycodeschool.ro)